

CXL-Vector: High-Throughput Graph-Based Vector Search on CXL Memory

Anonymous Author(s)

Abstract

Graph-based ANNS is fundamentally a memory hierarchy problem: each query performs fine-grained, data-dependent graph traversal whose throughput depends on fast random access to the index. However, as vector datasets scale, raw vectors become the dominant memory consumer, exceeding the DRAM capacity of a single server. Existing SSD-based solutions expand capacity but mismatch the random access pattern of graph traversal, incurring substantial block-I/O overhead. CXL memory offers capacity expansion beyond DRAM while preserving byte-addressable memory access, yet existing approaches remain limited: hardware-assisted CXL acceleration requires specialized endpoint support, while naively placing the entire index in CXL memory saturates the shared bandwidth under concurrent graph search. We present CXL-Vector, a pure-software framework for high-throughput graph-based ANNS on commodity CXL-enabled servers. The key idea is to treat CXL memory as a bandwidth-bounded capacity tier rather than as slower DRAM. CXL-Vector introduces a tier-aware execution model that reorganizes graph search across memory tiers while preserving high throughput on commodity CXL systems. It further restructures graph expansion into batched execution and applies a vectorized distance kernel over compact representations, exposing parallelism without graph reconstruction or replicated index layouts. Finally, CXL-Vector introduces scalable query execution mechanisms to sustain high concurrency by reducing memory-access stalls, address-translation overhead, and visited-set contention. Extensive experiments show that at Recall@10 ≥ 0.90 , CXL-Vector achieves 3.9 $\times$ –7.2 $\times$ higher throughput than HNSW while reducing local DRAM usage by 58%. It outperforms SymphonyQG by 1.8 $\times$ –2.9 $\times$ with 76%–78% less local DRAM and maintains a 29.1 $\times$ throughput advantage over DiskANN-SSD at billion scale. Relative to placing the entire index in CXL memory, CXL-Vector reduces CXL read bandwidth by 14.6 $\times$ –563.3 $\times$.

1 Introduction

Approximate nearest-neighbor search (ANNS) has become a fundamental primitive for modern vector search workloads [39, 43, 46], including retrieval-augmented generation (RAG), recommendation, semantic search, and large-scale information retrieval [15, 23, 33, 40, 52]. Among existing approaches [5, 10, 18, 26], graph-based indexes such as HNSW and NSG dominate practical deployments because

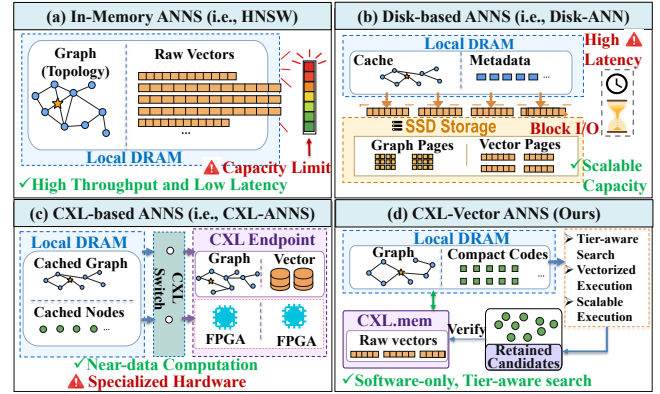


Figure 1: Placement choices for graph-based ANNS indexes.

they provide an attractive tradeoff between search accuracy and throughput [4, 13, 36, 54]. However, scaling graph-based ANNS remains challenging. A graph index must jointly maintain graph connectivity and high-dimensional vector representations, where raw vectors often dominate the memory footprint [6, 8, 11, 48, 53]. Consequently, large-scale deployments rapidly exceed the DRAM capacity of a single server, making memory capacity a primary obstacle to scaling graph-based ANNS.

Existing graph-based ANNS systems adopt different index placement strategies to balance capacity and performance, as summarized in Figure 1. Conventional DRAM-resident graph indexes [4, 13, 36, 54] keep the entire index in local DRAM (Figure 1(a)), providing high throughput but limiting scalability to available memory capacity. To overcome this capacity limitation, SSD-based systems such as DiskANN, SPANN, and Starling [6, 8, 48, 53] introduce SSDs as a capacity tier (Figure 1(b)), retaining only lightweight routing structures in DRAM while storing large portions of the index on SSD. This substantially expands index capacity, but graph traversal incurs high-latency block I/O and read amplification because its fine-grained random accesses are translated into coarse-grained page reads. More recently, CXL-based ANNS systems leverage near-data processing at CXL endpoints to perform vector-distance computation close to memory (Figure 1(c)) [27]. Although this design reduces host–CXL data movement, it relies on specialized endpoint-side acceleration rather than commodity memory-only CXL systems, leaving the potential of CXL memory expansion for software-based graph ANNS unexplored.

Commodity CXL memory expansion opens a new opportunity for scaling graph-based ANNS. Unlike SSDs, CXL extends the host memory hierarchy with byte-addressable load/store access, allowing graph indexes to scale through load/store access rather than block-based I/O. This makes CXL an attractive capacity tier for commodity servers. However, naively relocating graph indexes from DRAM to CXL does not preserve their performance [27, 49].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGMOD '27, Huntington Beach, CA, USA
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

Existing graph indexes [40, 52, 58] are optimized for a uniform-memory hierarchy and repeatedly access graph records and raw vectors throughout graph traversal. When the entire index is placed in CXL memory (*all-CXL placement*), every intermediate candidate incurs a CXL read to access its full-precision vector, although only a small fraction of visited nodes are retained in the final result. Under concurrent queries, these fine-grained random reads accumulate rapidly and saturate the shared CXL read bandwidth. Our placement study (§2.2) confirms this behavior: *all-CXL placement* reduces throughput by 12.5× under concurrency, shifting the bottleneck from memory capacity to memory access efficiency.

Consequently, the challenge is no longer where to place the graph index, but how to reorganize graph search for a tiered memory hierarchy. Such a reorganization must simultaneously address three coupled challenges:

C1. Decoupling Traversal from Capacity-Tier Accesses: Graph-based ANNS systems are typically optimized for a uniform memory hierarchy, where graph traversal and vector access share the same memory tier [13, 36, 40, 52, 58]. However, naively placing the entire index in CXL memory transforms capacity scaling into a bandwidth bottleneck: concurrent graph traversals generate fine-grained random reads that contend for the shared CXL bandwidth. The challenge is to reorganize index layout and search execution for a hierarchical memory system, preserving traversal efficiency and recall without placing raw-vector accesses.

C2. Vectorization without Index Inflation: Many modern ANNS systems reduce traversal-time computation by using compact quantized representations [11, 48, 52]. However, efficiently processing these compact codes with SIMD remains challenging because graph traversal exposes candidates in an irregular manner. Existing SIMD-oriented graph-search techniques address this issue by replicating neighbor codes along graph edges or introducing auxiliary memory layouts to increase data parallelism [2, 19, 28]. These approaches improve computation efficiency but significantly enlarge the local-memory footprint, reducing the capacity benefits of CXL-tiered indexing. The challenge is to expose vector-level parallelism during graph search without replicating the index.

C3. Memory-Hierarchy Contention under Concurrency: Graph traversal performs irregular memory accesses whose cost increases with query concurrency due to contention in the host memory hierarchy [1, 37]. Under CXL-tiered memory, these overheads are further amplified by the higher latency of capacity-tier accesses [9, 27]. The third challenge is to sustain scalable query execution despite the combined pressure across the hierarchical memory system.

To address these challenges, we present CXL-Vector, a pure-software execution framework for graph-based ANNS on commodity servers equipped with CXL memory expansion (Figure 1(d)). Rather than redesigning existing graph indexes, CXL-Vector reorganizes their execution around a hierarchical memory system while preserving their graph structures. Its three tightly integrated components respectively address the challenges above and constitute the main contributions of this paper:

- **Tier-Aware Graph Search (C1).** Graph search is reorganized around a tier-aware index layout that decouples traversal from capacity-tier vector accesses. Graph topology and compact vector representations remain in local DRAM, allowing traversal to execute entirely within the host memory hierarchy, whereas

raw vectors reside in CXL memory and are accessed only for final candidate verification. An explicit reranking budget limits final verification to a bounded set of retained candidates while preserving high recall (§4.1).

- **Replication-Free Batched Execution (C2).** CXL-Vector restructures the graph traversal to recover vectorization opportunities without inflating the graph index. Instead of replicating compact codes into SIMD-friendly layouts, it decouples neighbor collection from distance evaluation, exposing each graph expansion as a batch of independent candidates. A unified vectorized distance kernel evaluates these batches over compact vector representations, exploiting both intra-vector and inter-candidate parallelism while avoiding code replication (§4.2).
- **Scalable Query Execution under Concurrency (C3).** CXL-Vector counters the contention that concurrent queries induce across the memory hierarchy: chunk-ahead prefetching hides memory-access stalls in the distance kernel, tier-local huge-page mappings cut address-translation overhead, and a cache-resident blocked Bloom filter curbs LLC contention from visited-set probes. Together, these mechanisms sustain high query throughput as concurrency increases (§5).

We implement CXL-Vector¹ on both HNSW and NSG without modifying their graph structures, demonstrating that the proposed execution model is applicable across different graph-based indexes. Extensive experiments show that CXL-Vector effectively exploits commodity CXL memory while preserving search accuracy. At matched recall, CXL-Vector achieves 3.9×–7.2× the throughput of HNSW with 58% less local DRAM, outperforms the SymphonyQG by 1.8×–2.9× with 76%–78% less local DRAM, and sustains a 29.1× throughput advantage over DiskANN-SSD at billion scale. Relative to placing the entire index in CXL memory, CXL-Vector cuts CXL read traffic by 14.6×–563.3× (§6).

2 Background and Motivation

2.1 The Raw-Vector Bottleneck

Graph-based ANNS repeatedly accesses graph records together with vector data during graph traversal to guide the search process [13, 36, 48]. Since these accesses lie on the critical path of query execution, high-performance implementations keep the entire index in local DRAM to avoid exposing traversal to higher-latency memory accesses [11, 52]. As vector collections continue to grow, however, this memory-first design fundamentally limits scalability to the available DRAM capacity. More importantly, the scalability of graph-based ANNS is fundamentally constrained not by graph organization, but by the raw-vector payload.

Unlike graph topology, whose footprint depends on the index design and construction parameters, the raw-vector payload is intrinsic to the underlying collection and scales directly with dataset size and embedding dimensionality. For a collection of N d -dimensional Float32 vectors, the raw-vector array alone occupies $4Nd$ bytes. In contrast, the graph stores only a bounded neighbor list per node, whose size is determined by the maximum out-degree and is independent of the embedding dimensionality. The memory footprint of raw vectors therefore grows linearly with both the collection size

¹To facilitate reproducibility, the source code and artifacts are available at: <https://anonymous.4open.science/r/artifact-052D/>

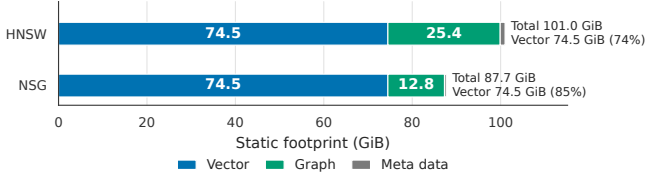


Figure 2: Native memory footprint on Text2Image100M.

Table 1: Local DRAM vs. CXL memory.

Property	Local DRAM	CXL memory
Idle read latency (ns)	80.7	192.3
Random-read BW, peak (GiB/s)	140.4	16.9
Random-read BW, 56 threads (GiB/s)	140.4	13.7

and the embedding dimensionality, whereas the graph overhead grows much more slowly. Consequently, although graph indexes adopt distinct graph structures, they all retain the same raw-vector payload. As datasets and embedding dimensions continue to grow, the scalability of graph-based ANNS becomes fundamentally constrained by raw vectors rather than the graph structure [8].

Figure 2 confirms this observation on the Text2Image100M dataset. Although HNSW and NSG employ substantially different graph organizations, raw vectors dominate the memory footprint in both cases, accounting for 74% and 85% of the total index size, respectively. More importantly, this invariant holds despite the different graph structures adopted by HNSW and NSG: only the graph component changes, whereas the raw-vector footprint remains identical because it is determined solely by the underlying dataset. Consequently, optimizing graph organization alone cannot fundamentally improve memory scalability. These observations suggest that improving the scalability of graph-based ANNS ultimately requires treating raw vectors as a capacity-tier resource. The remaining challenge is therefore not *what* to offload, but *how* to reorganize graph search for a hierarchical memory system while preserving search throughput.

2.2 Limitations of Transparent CXL Placement

Existing systems expand capacity beyond local DRAM either by scaling byte-addressable memory at high cost, through larger-memory servers or RDMA-based distributed indexes [7, 12, 60], or by offloading the index to SSDs [6, 20, 32, 48, 53], where the fine-grained, data-dependent accesses of graph traversal meet page-granular I/O and incur read amplification on the critical search path.

CXL memory expansion provides a promising opportunity for scaling graph-based ANNS. Unlike SSDs, it extends the host memory hierarchy with cache-coherent, byte-addressable load/store access while providing substantially larger capacity than local DRAM [9, 49, 56]. This combination makes CXL a natural capacity tier for graph indexes, suggesting a straightforward design that transparently places the entire graph index in CXL memory while preserving existing graph-search algorithms.

However, this intuition breaks down under high query concurrency. Graph-based ANNS achieves high throughput by executing many queries concurrently across CPU worker threads. Under transparent CXL placement, every graph expansion fetches graph records and raw vectors from the shared CXL memory tier.

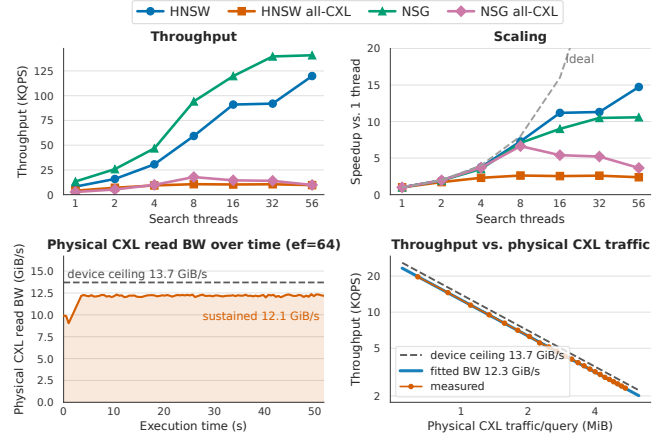


Figure 3: All-CXL placement on SIFT10M: throughput scaling (top) and CXL read bandwidth (bottom).

Although CXL provides sufficient capacity, the aggregate traffic generated by concurrent traversal rapidly saturates the shared CXL read channel, fundamentally shifting the bottleneck from memory capacity to CXL read bandwidth. This trend is consistent with the hardware characteristics of our platform (Table 1), where a single CXL device exhibits 2.4 $\times$ higher read latency, 8.3 $\times$ lower random-read bandwidth, and 10.2 $\times$ lower bandwidth under 56-thread contention.

To quantify this effect, we conduct a controlled placement study on SIFT10M, where the complete graph index fits in either local DRAM or CXL memory. We compare two configurations under identical search parameters: (1) **native DRAM placement** and (2) **transparent all-CXL placement**, in which the entire graph index resides in CXL memory.

The upper panels of Figure 3 report query throughput as concurrency increases. Native DRAM placement continues to scale with thread count for both HNSW and NSG, whereas all-CXL placement quickly reaches a throughput plateau beyond which additional worker threads provide little benefit. At matched recall and 56 threads, transparent CXL placement reduces throughput by 12.5 $\times$ for HNSW and 14.4 $\times$ for NSG, despite executing the same graph-search algorithm over the same graph indexes.

The underlying cause is the excessive CXL traffic generated during graph traversal. Each graph expansion evaluates a large number of intermediate candidates, most of which are eventually discarded before the final top- K results. Nevertheless, under transparent placement every candidate evaluation requires reading CXL-resident vectors, causing CXL traffic to scale with traversal complexity rather than the number of final results. As concurrency increases, these redundant accesses collectively saturate the shared CXL read channel, preventing throughput from scaling.

The lower panels of Figure 3 confirm this diagnosis. We quantify controller-observed CXL read traffic using the CXL Cache/Mem uncore PMU.² Across different search widths, controller-observed

²We count `UNC_CXLCM_RxC_PACK_BUF_INSERTS.MEM_DATA` with the raw event `cxlcm/event=0x41,umask=0x10/` under `perf stat -a` and convert each `MEM_DATA` packing-buffer insertion to 64 B, consistent with Intel PCM’s conversion for this event. We read the raw PMU because PCM does not expose the per-port read metric on this Emerald Rapids platform.

CXL traffic per query varies by nearly 9×, yet the observed bandwidth remains nearly constant at 12.2–12.4 GiB/s, approximately 90% of the device peak bandwidth. Consequently, query throughput decreases almost inversely with the CXL bytes consumed per query, identifying shared CXL read bandwidth as the throughput constraint.

These observations lead to an important conclusion. Transparent CXL placement fails not because CXL memory is simply slower than local DRAM, but because conventional graph traversal generates CXL traffic proportional to traversal complexity, quickly exhausting the shared read bandwidth under concurrent execution. Consequently, efficiently exploiting CXL memory requires reorganizing graph search for a hierarchical memory system, removing raw-vector accesses from traversal and confining them to bounded final verification.

3 System Overview

CXL-Vector is founded on the observation that graph traversal and exact similarity computation exhibit fundamentally different memory-access characteristics. Rather than redesigning existing graph indexes, CXL-Vector reorganizes graph execution over a hierarchical memory system, allowing each stage to execute on the memory tier best suited to its access pattern while preserving the original graph structures. Figure 4 illustrates the overall architecture. A pool of worker threads concurrently executes graph search through a shared execution framework, where graph traversal operates entirely on DRAM-resident graph data and exact distance calculation accesses CXL-resident raw vectors only during the final reranking stage.

Tier-Aware Graph Search. CXL-Vector reorganizes graph search around a hierarchical memory system by jointly designing index organization and traversal. Graph topology together with compact vector representations remains in local DRAM, while raw vectors reside in CXL memory and are accessed only during bounded reranking. This organization decouples graph traversal from capacity-tier raw-vector accesses while preserving high-recall search.

Replication-Free Batched Execution. Tier-aware search removes raw-vector accesses from graph traversal but does not fully exploit modern CPUs because candidate evaluation remains largely sequential. CXL-Vector therefore restructures graph search execution to expose batches of independent candidates and processes them through a unified vectorized distance kernel. Unlike existing replication-based designs [19, 28], this batched execution recovers both intra-vector and inter-candidate parallelism without replicating compact codes or modifying the graph structure.

Scalable Query Execution. Finally, CXL-Vector sustains high query throughput as worker concurrency grows. Chunk-ahead prefetching, huge-page mappings, and a cache-resident blocked Bloom filter collectively reduce irregular-access stalls, address-translation overhead, and visited-set contention.

4 Graph Search Engine

4.1 Tier-Aware Search

To exploit CXL memory as a capacity tier without exposing graph traversal to CXL accesses, CXL-Vector decouples traversal from

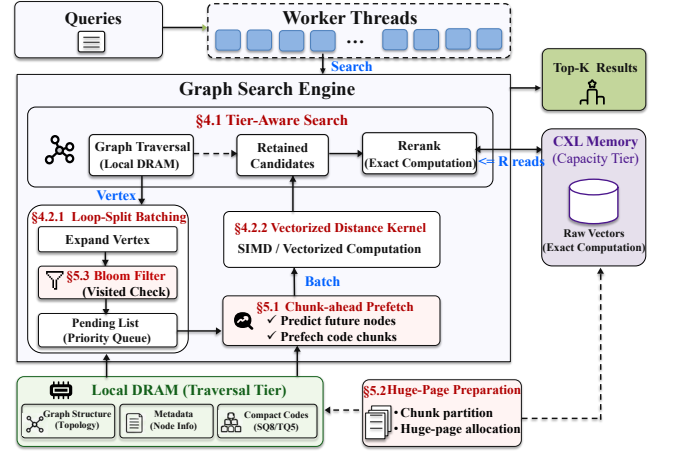


Figure 4: CXL-Vector system architecture.

raw-vector accesses. Graph traversal is guided entirely by compact vector representations in local DRAM, while raw vectors are accessed only during a bounded reranking stage in CXL memory. This separation confines capacity-tier raw-vector access to final verification rather than every traversal step.

Two-Tier Index Layout and Compact Codes. CXL-Vector organizes the index into two memory tiers based on the role of each component during graph search. The graph structure and a fixed-size compact code for every node are placed in local DRAM, allowing each graph expansion to be completed entirely from local memory. During traversal, neighbor lists are retrieved from the graph structure, while approximate distances are computed from the corresponding compact codes. The capacity-dominant raw vectors reside in CXL memory and are accessed only during reranking. This organization removes CXL accesses from the critical path of graph traversal while preserving compatibility with existing graph indexes: HNSW retains its hierarchical graph, and NSG retains its monotonic search graph without modifying their graph topologies [13, 36].

Compact codes enable this organization by replacing raw vectors during traversal. Each code is a fixed-size lossy representation that is small enough to co-locate with the graph in DRAM while preserving sufficient fidelity to guide graph traversal. CXL-Vector is agnostic to the specific code format and supports both SQ8 (8-bit scalar quantization) [11] and TQ5 (5-bit TurboQuant) [57]. SQ8 offers lower computational overhead, whereas TQ5 provides a smaller memory footprint. Importantly, compact codes need only preserve the traversal order rather than exact distances, since the retained candidates are re-evaluated using full-precision vectors during reranking.

Approximate Distances During Traversal. Although the two-tier layout confines raw vectors to CXL memory, conventional best-first search still depends on them during traversal. In standard graph search (Algorithm 2, Appendix A), every newly discovered node consults its full-precision vector to determine candidate ordering before graph expansion. Each graph expansion therefore incurs multiple CXL accesses, making CXL traffic proportional to the traversal complexity and eventually saturating the shared CXL bandwidth under concurrent queries.

Rather than relying on exact distances throughout graph traversal, CXL-Vector guides traversal using an *approximate* distance $\tilde{D}(q, v)$ computed from the node's compact code in local DRAM, while deferring the exact distance $D(q, v)$ to a bounded reranking stage over the retained candidates. The approximate distances determine candidate ordering and graph expansion, whereas exact distances are computed only for the retained candidates. Approximate traversal followed by exact refinement is a well-established strategy in quantization-based ANNS systems [11, 28, 48]. CXL-Vector repurposes this approximation-refinement separation as a memory-system boundary, keeping all traversal-time accesses within local DRAM while confining CXL-resident raw-vector access to final reranking.

Bounded Reranking. Only after graph traversal terminates does CXL-Vector access CXL memory to restore the exact ordering of the retained candidates. Reranking selects the R candidates with the smallest approximate distances, retrieves their raw vectors from CXL memory, recomputes the exact distance $D(q, v)$, and returns the top- K results. Since $R \geq K$, reranking operates on a candidate set that is no smaller than the final result set. Exact distances may reorder the retained candidates but cannot recover candidates that have already been pruned during traversal. Consequently, the reranking budget R bounds the number of raw-vector records selected for exact evaluation. The corresponding cache-line demand footprint and controller-observed physical traffic are distinguished below.

CXL record selection and cache-line demand footprint. Let C_q be the candidates selected for reranking and $m_q = |C_q| = \min(R, |S|)$. CXL-Vector selects exactly m_q raw-vector records for exact distance evaluation and selects none during traversal. This record-count bound, $m_q = \min(R, |S|) \leq R$, is independent of cache-line alignment.

Converting this count into a cache-line footprint requires accounting for the lines covering each record. On our platform, cache lines are 64 B. Let $r_d > 0$ be the length of the contiguous CXL-resident byte span touched by one exact-vector evaluation, including any padding or vectorized over-read accessed by the kernel; let a_v be the span's starting address and $\delta_v = a_v \bmod 64$ its offset within a cache line. If $\mathcal{L}(v)$ is the set of cache lines intersecting $[a_v, a_v + r_d)$, then that record's cache-line footprint is $r_{CL}(v) \equiv 64|\mathcal{L}(v)| = 64\lceil(r_d + \delta_v)/64\rceil$. Since $0 \leq \delta_v \leq 63$, each record's footprint is bounded by $r_{CL}^{\max} \equiv 64\lceil(r_d + 63)/64\rceil$. Hence, the distinct cache-line footprint $F_{CL}(q)$ of the records selected for query q satisfies

$$F_{CL}(q) \leq \sum_{v \in C_q} r_{CL}(v) \leq m_q r_{CL}^{\max} \leq R r_{CL}^{\max}. \quad (1)$$

Equation (1) bounds the selected records' cache-line demand footprint. Controller-observed physical traffic, b_q , may be lower than this footprint because of cache hits or higher because of prefetches and refills. Once $R \geq |S|$, all retained candidates are selected and the footprint stops growing.

CXL-Bandwidth Ceiling. To relate physical traffic to the CXL bandwidth ceiling, let b_q be the *controller-observed* physical CXL read traffic per completed query, $\mathcal{B}_{CXL}$ the measured 56-thread random-read saturation bandwidth (13.7 GiB/s), and λ the query throughput. Their product λb_q is the controller-observed bandwidth

consumed by the workload and cannot exceed $\mathcal{B}_{CXL}$:

$$\rho_{CXL} \equiv \frac{\lambda b_q}{\mathcal{B}_{CXL}}, \quad \lambda \leq \frac{\mathcal{B}_{CXL}}{b_q}. \quad (2)$$

Here ρ_{CXL} is the controller-observed CXL-bandwidth utilization. When $\rho_{CXL} \approx 1$, aggregate CXL bandwidth is the active bottleneck and $\lambda \approx \mathcal{B}_{CXL}/b_q$; when $\rho_{CXL} \ll 1$, that quotient is only an unattained bandwidth ceiling.

The cost of reranking also depends on the access granularity of the capacity tier. Since retained candidates may reside at arbitrary addresses, reranking issues scattered, independent raw-vector fetches that incur severe read amplification on block-addressable storage such as SSDs: each fetch transfers a full page to retrieve a single vector [48, 53]. CXL's byte-addressable load/store interface avoids this overhead: each raw vector is fetched at cache-line granularity, transferring only the bytes requested.

Recall Implications. Approximate traversal affects recall through two distinct mechanisms. First, approximate distances may alter traversal decisions, causing relevant candidates to be pruned before reranking. Since these candidates are never retained, reranking cannot recover them; this loss can only be reduced by improving traversal quality, e.g., using higher-fidelity compact codes or a larger search width L . Second, among the retained candidates, some true neighbors may not be selected for reranking because they fall outside the top- R according to approximate distances. This loss decreases monotonically as the reranking budget R increases and disappears once $R \geq |S|$, where every retained candidate receives an exact distance. Consequently, increasing R eliminates only reranking-induced recall loss, whereas any remaining loss is attributable solely to approximate traversal.

4.2 Batched Graph Search Execution

The tier-aware search of Section 4.1 reorganizes graph search for tiered memory by confining traversal to local DRAM and restricting raw-vector access to final reranking over at most R candidates. Compared with transparent CXL placement, it eliminates most of the performance penalty, reducing the throughput gap from a 12.5× slowdown to roughly 2× relative to a native DRAM-resident HNSW at matched recall (Figure 5a). Bridging the remaining gap requires optimizing query execution rather than changing the tier-aware search algorithm. This motivates a batched execution that preserves the same search strategy while improving throughput.

Root Cause: Underutilized Hardware. A profile of tier-aware search identifies the source of this slowdown. Figure 5b breaks search time down by operation: distance evaluation is the largest single component, accounting for 50%–55% of search time across all five datasets. This dominant cost is shaped by the structure of the traversal loop. The conventional loop is *fused*: for each neighbor, it computes the distance, tests it against $\text{Worst}(\mathcal{S})$ (the farthest candidate in the retained pool $\mathcal{S}$), and updates the insertion threshold before turning to the next. The per-neighbor distances are mutually independent, yet the fused loop never exposes them together, leaving the CPU's wide vector units idle.

Unexploited Parallelism in Graph Expansion. Existing quantization augmented indexes address SIMD underutilization through index-layout redesign: NGT-QG and SymphonyQG [19, 28] store a

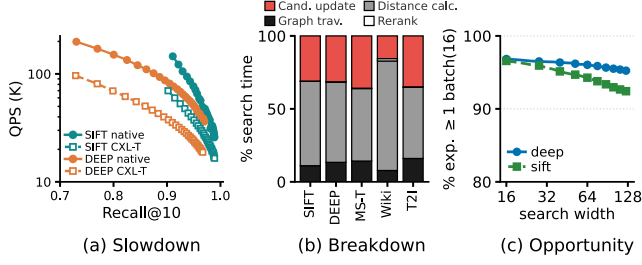


Figure 5: (a) Tier-aware search (CXL-T) trails native in-DRAM search at matched recall. (b) Distance calculation dominates search-phase time. (c) Graph expansion almost always exposes a full batch.

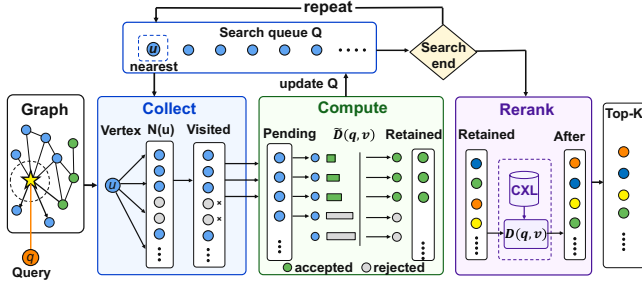


Figure 6: Staged query execution in CXL-Vector: graph expansion over compact codes, batched distance evaluation, and bounded reranking with CXL-resident raw vectors.

copy of each neighbor’s compact code alongside the vertex’s neighbor list in a contiguous per-vertex block, so that FastScan [2] can scan all neighbors in a single pass. This recovers parallelism but at two costs:

- (1) Each vertex block stores a copy of every neighbor’s code. Ignoring padding and per-code metadata, at average degree $\bar{m}$, the replicated-code payload occupies $\bar{m}b/32$ times the Float32 raw vectors, where b is the number of code bits per coordinate,³ an inflation that falls on the local-DRAM usage we aim to reduce.
- (2) FastScan computes distances for all neighbors of a visited vertex, even those already seen, incurring even more of the computation that already dominates search time.

The key observation is that graph traversal already exposes abundant parallelism without code replication. A graph expansion rarely reveals only one unseen neighbor. Instead, traversing a neighbor list naturally produces a group of previously unseen neighbors whose distances are independent and can therefore be evaluated together. Figure 5c confirms that this opportunity is pervasive: over 90% of graph expansions expose at least B unseen neighbors ($B=16$ by default) across all evaluated search widths. Although wider searches increase overlap with the visited set $\mathcal{V}$, enough unseen neighbors remain to sustain batched distance evaluation throughout traversal.

4.2.1 Loop-Split Batching. Two-Pass Split. To expose the parallelism identified in graph expansion, CXL-Vector decouples graph traversal from distance evaluation by splitting the conventional traversal loop into two passes (Figure 6). The *collect* pass performs

³SymphonyQG replicates 1-bit RaBitQ codes at out-degree 32, already matching the raw vectors ($32 \cdot 1/32 = 1\times$); replicating the 8-bit SQ8 codes we adopt would cost $8\times$ as much [19].

Algorithm 1 Tier-aware graph search with loop-split batching

Require: query q , result size K , search width L
Require: reranking budget $R \geq K$, batch size B

- 1: initialize $Q, S, \mathcal{V}$ from entry point $\text{Entry}(q)$
- 2: **while** $Q \neq \emptyset$ **do**
 Collect: expand one node; no raw-vector read
 $u \leftarrow \text{Best}(Q)$
 if $|S| = L$ and $\tilde{D}(q, u) > \text{Worst}(S)$ **then break**
 end if
 $Q \leftarrow Q \setminus \{u\}$
 $P \leftarrow [v \in N_G(u) : v \notin \mathcal{V}]$; $\mathcal{V} \leftarrow \mathcal{V} \cup P$
 Compute and insert: approximate distances
 for all batch C of $\leq B$ candidates from P **do**
 $\tilde{D}(q, v) \leftarrow$ vectorized distance computation for all $v \in C$
 insert survivors into Q and S ; trim S to L
 end for
- 3: **end while**
 Rerank: exact evaluation of at most R raw-vector records
 13: compute $D(q, v)$ from CXL for closest $\min(R, |S|)$ entries
 14: **return** K entries with smallest $D(q, v)$

only graph expansion: it expands the selected node, filters out visited neighbors, and appends the remaining candidates to a pending list P . The *compute-and-insert* pass then processes P in batches of size B , computes approximate distances from compact codes, and inserts qualified candidates into both the retained pool S and the priority queue Q . The pending list adapts to the number of newly discovered neighbors, whereas the batch size is fixed to match the execution granularity. By separating graph expansion from candidate evaluation, loop-split batching exposes independent work without replicating index data or changing the search algorithm.

Algorithm 1 formalizes the loop-split execution. The priority queue Q maintains discovered candidates ordered by approximate distance, with $\text{Best}(Q)$ returning the closest one. Each iteration first performs the *collect* stage, which expands the selected node and appends all previously unseen neighbors to the pending list P . The *compute-and-insert* stage then processes P in successive batches of size B , computes approximate distances from compact codes, and inserts candidates satisfying the search criterion into both the priority queue Q and the retained pool S . After traversal terminates, the *rerank* stage performs the bounded reranking of Section 4.1 to produce the final top- K results.

Termination. Two conditions end the loop. First, the queue empties. Second, S has reached capacity L and the nearest queued node already exceeds $\text{Worst}(S)$ under the approximate distance, so no remaining candidate can improve the pool.

4.2.2 Vectorized Distance Kernel. Loop-split batching exposes independent candidates during graph expansion, but efficient execution still requires high-throughput distance computation. Compact codes reduce the memory footprint and data movement of each candidate, yet scalar decoding and coordinate-wise computation limit computational efficiency. Existing quantization-aware indexes improve distance evaluation through specialized layouts and replicated codes [2, 3, 11, 19]. CXL-Vector instead exploits the batches naturally exposed by graph traversal and implements a vectorized distance kernel that operates directly on compact codes, achieving

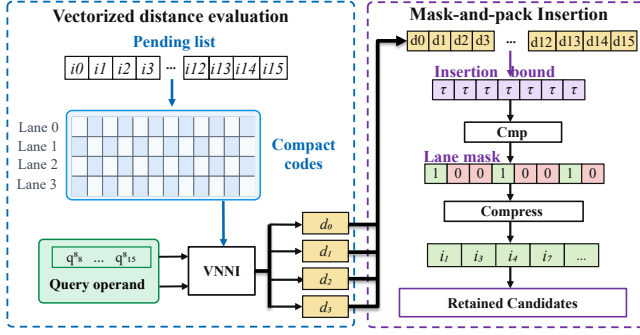


Figure 7: Vectorized distance kernel.

high-throughput execution without modifying the graph layout or increasing the DRAM footprint.

The kernel exploits the shared computation pattern between squared-Euclidean and maximum-inner-product search. Let $\hat{x}_v$ denote the reconstruction represented by candidate v 's compact code:

$$\begin{aligned}\tilde{D}_{L2}(q, v) &= \|q - \hat{x}_v\|_2^2 = \|q\|_2^2 + \|\hat{x}_v\|_2^2 - 2\langle q, \hat{x}_v \rangle, \\ \tilde{D}_{IP}(q, v) &= -\langle q, \hat{x}_v \rangle.\end{aligned}\quad (3)$$

For both metrics, the query-dependent computation reduces to an inner product between the query and the compact-code representation; norm terms and quantizer-specific offsets are handled as lightweight per-vector corrections. CXL-Vector therefore implements a unified integer dot-product kernel using AVX-512 VNNI (vpdpbusd), which fuses byte-level multiplication and accumulation in a single instruction. The same kernel supports both SQ8 and TQ5 representations and both distance metrics, with only the final correction step differing across configurations.

Intra- and inter-candidate parallelism. The left panel of Figure 7 illustrates batch execution in the kernel. The kernel exploits the parallelism exposed by loop-split batching at two levels. *Within* a candidate, the VNNI instruction multiplies and accumulates 64 coordinates per invocation, computing each distance in wide steps rather than one coordinate at a time. *Across* candidates, the kernel evaluates four candidates concurrently, so several dot products are always in flight. This batching is what keeps the vector units busy: a single candidate forms one dependency chain and stalls on the multi-cycle latency of each vpdpbusd, whereas a batch supplies enough independent work to overlap those latencies and increase kernel throughput by overlapping independent dependency chains. The query operand is constructed once per query and reused across all candidates in the batch. After processing a batch, the kernel outputs approximate distances aligned with the corresponding entries in the pending list.

Quantizer adaptation. CXL-Vector uses the same vectorized kernel for different compact-code formats. While SQ8 naturally stores each coordinate as an unsigned byte and can directly feed the integer dot-product path, TQ5 represents each coordinate as a compact codebook index. To bridge this representation gap, CXL-Vector re-quantizes TQ5 codebook entries into unsigned 8-bit values, allowing packed indices to be converted into the byte operands required by the unified kernel through lightweight in-register lookups.

Table 2 quantifies the benefit of this codebook re-quantization. The floating-point baseline decodes each 5-bit index to its original

Table 2: Measured per-block compute cost of the two TQ5 distance kernels on Emerald Rapids.

Kernel	cyc/blk	IPC	μ ops/blk	port-5/blk
TQ5 (integer)	8.01	4.14	32.4	8.8
TQ5 (fp decode)	157.4	0.82	128.0	30.8

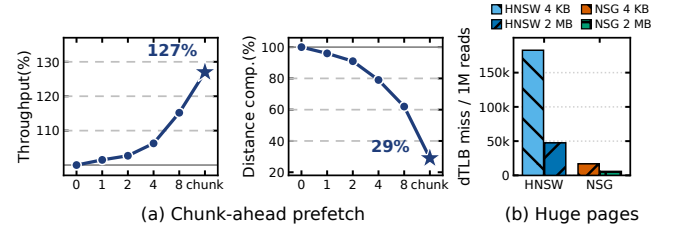


Figure 8: Chunk-ahead prefetching and huge pages.

codebook entry and computes the distance in float. The integer kernel, by contrast, is roughly 20 $\times$ faster per coordinate block (64 coordinates), executing about 4 $\times$ fewer micro-ops at roughly 5 $\times$ the IPC. The gap arises because the floating-point path requires a per-coordinate codebook lookup that saturates the shuffle port. Quantizing the codebook to bytes relieves this pressure: the smaller entries allow more lookups per shuffle instruction, and the post-lookup computation reduces to a single VNNI multiply-accumulate, largely removing the shuffle-port pressure.

Mask-and-pack insertion. After vectorized distance computation, the generated distance array must be filtered before the corresponding candidates enter the retained pool S . Since graph search rejects the vast majority of candidates, a scalar admission loop suffers frequent branch mispredictions. The mask-and-pack step (Figure 7, right) filters branchlessly instead [44]: once $|S|$ reaches the search width L , CXL-Vector broadcasts $\text{Worst}(S)$, compares it against the full distance array in a single SIMD pass, and compresses the accepted distances and identifiers into dense arrays via the output lane mask. The SIMD filtering step uses the current value of $\text{Worst}(S)$ as the admission threshold at the beginning of each batch. However, this threshold can change during batch insertion: as better candidates from the same batch enter S , the farthest retained candidate may be replaced and the threshold becomes more restrictive. Therefore, CXL-Vector rechecks each packed candidate against the updated threshold before inserting it into Q and S . Consequently, batching changes only how distances are computed, not which candidates enter Q and S or when traversal terminates.

5 Scalable Query Execution under Concurrency

By confining traversal-time accesses to local DRAM, CXL-Vector avoids the CXL bandwidth bottleneck caused by transparent CXL placement. However, high query concurrency exposes remaining limitations in the host memory hierarchy. Profiling identifies three dominant factors: irregular memory accesses during compact-code evaluation, address-translation overhead from scattered graph traversal, and cache contention from concurrent visited-set operations. Here we present three mechanisms—Chunk-Ahead Prefetching, Huge-Page Preparation, and Cache-Resident Bloom Filter—to alleviate these host-side inefficiencies and sustain throughput scaling.

5.1 Chunk-Ahead Prefetching

Existing graph ANNS implementations employ software prefetching to hide memory latency during traversal [13, 36]. However, these techniques rely on two assumptions: future accesses are predictable from the current neighbor list, and computation on the current candidate provides sufficient latency-hiding opportunity for the prefetched data. Neither assumption holds for CXL-Vector’s batched execution. Compact-code distance computation is substantially shorter than a full-precision distance calculation, leaving insufficient computation to overlap the next memory access. Moreover, conventional traversal discovers candidate addresses incrementally and therefore cannot prefetch beyond the next neighbor.

Loop-split batching (§4.2.1) changes this execution model by materializing an entire pending list before distance computation. This exposes multiple future compact-code addresses and enables prefetching beyond a single neighbor. CXL-Vector exploits this visibility through *chunk-ahead prefetching*: while processing the current candidate batch, it prefetches compact codes for the next batch (16 candidates by default) and the neighbor list of the next node to expand.

Figure 8(a) evaluates the impact on Deep100M at 56 threads. Conventional single-neighbor prefetching provides only limited improvement because its lookahead distance is constrained by the traversal loop. In contrast, chunk-ahead prefetching reduces per-query compact-code access time by approximately 3.5× and improves throughput to 1.27× of the no-prefetch baseline. The gain comes from exploiting the batch-level visibility introduced by loop-split execution, which enables deeper lookahead than conventional neighbor-level prefetching.

5.2 Huge-Page Preparation

Graph traversal over large ANNS indexes suffers from address-translation overhead because neighbor expansions generate irregular accesses over a large memory footprint. With conventional 4 KB pages, the number of active page translations quickly exceeds dTLB capacity, causing frequent page walks [1, 37]. This overhead becomes more pronounced in CXL-Vector: although traversal remains in local DRAM, the graph layout and compact-code storage still generate scattered accesses, while CXL-resident raw vectors introduce additional translation overhead on the reranking path. Since CXL accesses already incur higher memory latency, avoiding unnecessary translation overhead becomes particularly important.

CXL-Vector therefore prepares the memory layout with huge pages before query execution. During index loading, it places each data segment on its target memory tier, requests 2 MB huge pages, and preallocates the allocated regions to complete page allocation and page-table construction offline. This preparation removes translation setup overhead from the online query path and provides a stable page mapping during concurrent search.

Figure 8(b) isolates the effect of huge pages on Deep100M using a graph-access replay that removes distance computation and reranking, exposing only pointer-chasing behavior. Replacing 4 KB pages with 2 MB pages reduces dTLB misses by 65%–74% across both graph backends at 56 threads, confirming that address translation is a significant source of concurrent traversal overhead and that huge pages effectively mitigate this bottleneck.

Table 3: Visited-set comparison on Deep100M.

Structure	QPS	R@10	LLC/q	miss%
Bloom filter	248K	0.8958	220	5.9
Generation array	169K	0.8969	2,734	39.2
Dense bitset	11.7K	0.8969	3,650	63.0

5.3 Cache-Resident Bloom Filter

Graph traversal requires a visited structure to avoid expanding the same node multiple times through different paths. Existing graph indexes typically maintain exact visited arrays or bitmaps whose size scales with the entire index. However, a query explores only a small fraction of the graph, making index-sized visited state inefficient under concurrent search. The problem becomes pronounced at large scale: for a billion-node index, a conventional visited array consumes about 2 GiB per query, leading to 104 GiB of DRAM solely for bookkeeping with 56 concurrent workers. Moreover, each lookup accesses a data-dependent location, causing LLC misses and additional contention among concurrent queries.

CXL-Vector replaces the index-sized exact structure with a *cache-resident blocked Bloom filter* [45]. The filter contains $m = 65,536$ bits (8 KB), divided into 128 cache-line-aligned 64 B blocks. Each inserted node sets $k = 2$ bits within one selected block, ensuring that every lookup touches only a single cache line. Under uniform hashing, the false-positive probability after inserting n nodes follows

$$p_{fp}(n) \approx (1 - e^{-kn/m})^k. \quad (4)$$

Unlike an index-sized visited structure, the memory footprint remains constant per query: concurrent execution with T workers consumes only $T \times 8$ KB of visited metadata, independent of index size. This cache-resident design targets LLC contention rather than memory capacity, making it complementary to CXL-Vector’s tier-aware index layout.

Table 3 compares visited structures on Deep100M at 56 threads with $L = 64$. The Bloom filter improves throughput by 1.47× at matched recall and reduces LLC misses per query by 12.4×. In contrast, the dense bitset suffers from large allocation and initialization overhead, while the exact generation-tagged array retains a large random-access footprint. The Bloom filter introduces only a small recall loss because false positives affect only candidate expansion; the impact remains limited under the chosen filter size.

6 Evaluation

We evaluate CXL-Vector to answer the following questions:

- **RQ1:** How does CXL-Vector compare with state-of-the-art baselines in throughput and resource usage while achieving the same recall (§6.2)?
- **RQ2:** Why is tier-aware search necessary for exploiting CXL memory, and why do transparent CXL placement and SSD-based tiering fail to scale (§6.3.1, 6.3.2)?
- **RQ3:** How much do tier-aware search and replication-free batching contribute to throughput (§6.3.3)?
- **RQ4:** What is the tradeoff between reranking budget, recall, and CXL traffic (§6.3.4)?
- **RQ5:** How does CXL-Vector’s scalable-execution design sustain throughput as concurrency increases (§6.3.5)?

6.1 Experimental Setup

Hardware. All experiments run on the same server using a single host NUMA node paired with one memory-only CXL NUMA node under Ubuntu 22.04.5. The server is equipped with an Intel Xeon Gold 5520+ CPU (28 cores, 56 threads), 576 GiB DDR5 DRAM, a 256 GiB CXL 1.1 memory expander over PCIe 5.0, and a 3.84 TB Intel D7-P5510 NVMe SSD.

Datasets. We evaluate CXL-Vector on six widely used ANN benchmarks spanning image, text, multimodal retrieval, and billion-scale search, covering both L2 and inner-product metrics (Table 4).

Baselines. CXL-Vector is instantiated on HNSW and NSG, denoted as *CXL-HNSW* and *CXL-NSG*. Both variants reuse the original graph structures and only modify the index representation and execution path required by CXL-Vector (Appendix D).

- **HNSW** [36]: representative in-memory graph index. We set $M=32$ and $ef_construction$ to 400.
- **NSG** [13]: in-memory graph index built on a monotonic search graph. We set $L=100$, $R=64$, and $C=400$.
- **Vamana** [48]: the in-memory graph index underlying DiskANN. We set $R=64$, $L_{build}=100$, and $\alpha=1.2$.
- **Faiss HNSW-Refine** [11]: HNSW augmented with SQ8 codes for quantized traversal and exact refinement. We set $M=32$ and $ef_construction$ to 400.
- **SymphonyQG** [19]: in-memory quantization-augmented graph index integrating RaBitQ codes [14]. We use degree bound 32, set EF_{build} to 400, and run three construction iterations.⁴
- **DiskANN-SSD** [48]: graph and raw vectors on SSD with PQ-compressed vectors in local DRAM. We set $R=64$, $L_{build}=100$, $\alpha=1.2$, and beam width $W=2$. The in-DRAM search budget is 64 GB, or 128 GB for BigANN.
- **CXL-ANNS-SW** [27]: software-only reproduction of CXL-ANNS over NSG, implementing its published caching and prefetching strategies.⁵

Metrics. Search quality is measured by Recall@10, and the performance is reported as throughput (QPS) under concurrent workloads. We also report the local-DRAM footprint, tier footprint (CXL memory for CXL-Vector/CXL-ANNS-SW and SSD for DiskANN-SSD), and per-query CXL traffic.

Default Settings. Unless otherwise specified, all throughput comparisons are performed at matched Recall@10. Worker threads are pinned to the host NUMA node after index loading. CXL-Vector uses SQ8 by default; the impact of quantizer choice is evaluated in Appendix C. Recall-QPS curves vary the search width of each method, while single-point studies fix the search width achieving 0.90 Recall@10 and vary only the mechanism under evaluation.

6.2 Overall Performance

Figure 9 evaluates whether CXL-Vector can simultaneously overcome the DRAM capacity constraint and preserve the throughput of in-memory graph indexes.

Throughput Analysis. CXL-Vector consistently achieves the highest throughput across all datasets at matched search quality. We

⁴We report SymphonyQG on the three L2 datasets and omit the two inner-product datasets to avoid a non-native distance-metric conversion.

⁵The original CXL-ANNS relies on endpoint-side acceleration unavailable on commodity memory-only CXL systems. We evaluate the software path to isolate its data-placement and caching effects.

Table 4: Characteristics of the evaluation benchmarks.

Benchmark	#Vectors	Dim.	Size (GiB)	Metric	Queries	Domain
SIFT	100M	128	47.7	L2	10K	Image
Deep	100M	96	35.8	L2	10K	Image
MSTuring	100M	100	37.3	L2	100K	Text
Wikipedia	35M	768	100.1	IP	5K	Text
Text2Image	100M	200	74.5	IP	100K	Multimodal
BigANN	1B	128	476.8	L2	10K	Image

compare methods at Recall@10 ≥ 0.90 by selecting the highest-QPS configuration of each method that satisfies this target. Under this setting, CXL-Vector improves throughput by $3.9\times$ – $7.2\times$ over HNSW, the representative DRAM-resident graph index, while reducing local DRAM usage. It also outperforms capacity-oriented designs by $17.9\times$ – $54.5\times$ over DiskANN-SSD and $34.8\times$ – $63.2\times$ over CXL-ANNS-SW, demonstrating that capacity expansion alone cannot address the bandwidth pressure of graph traversal. Compared with SymphonyQG, a quantization-augmented graph index, CXL-Vector achieves $1.8\times$ – $2.9\times$ higher throughput on the evaluated L2 datasets without replicating compact codes. The gains are consistent across graph structures: CXL-HNSW achieves the best throughput on three of five datasets, while CXL-NSG leads on SIFT and MSTuring. This demonstrates that the improvement comes from CXL-Vector’s memory-aware search and execution design rather than a specific index. DiskANN-SSD remains limited because each query incurs page-granularity SSD accesses, while CXL-ANNS-SW lacks the endpoint-side acceleration available in the original design and is constrained by commodity CXL bandwidth.

Recall Preservation. Despite using compact codes during traversal, CXL-Vector maintains competitive recall across all datasets. At the highest tested search width, its peak Recall@10 is within 0.005–0.023 of the strongest baseline, confirming that bounded reranking effectively compensates for most quantization error. The remaining gap comes from traversal-side candidate pruning, where approximate distances may alter the search path before reranking, as analyzed in Section 4.1. On the three L2 datasets, CXL-Vector also achieves higher peak recall than SymphonyQG by 0.005–0.046, whose more aggressive 1-bit RaBitQ quantization limits recall, especially on MSTuring.

Memory Allocation and Placement. We examine how CXL-Vector redistributes index components across memory tiers. Figure 9 decomposes the index allocation into local DRAM, CXL memory, and SSD capacity. By moving the capacity-dominant raw vectors to CXL while keeping graph structures and compact codes in local DRAM, CXL-Vector reduces local DRAM usage by 49%–71% compared with HNSW, 43%–72% compared with NSG, and 76%–78% compared with SymphonyQG. The reduction comes from avoiding both raw-vector storage and neighbor-code replication in DRAM. CXL-ANNS-SW and DiskANN-SSD achieve smaller local DRAM footprints through aggressive offloading, but rely on different execution models: the former requires endpoint-side acceleration, while the latter incurs block-oriented I/O overhead during search.

Billion-Scale Evaluation. Figure 10 evaluates CXL-Vector on BigANN with one billion uint8 vectors. The dataset occupies 119.2 GiB, fitting in the 256 GiB CXL expander and allowing an in-memory uint8 HNSW baseline within the 576 GiB DRAM budget. CXL-Vector places graph structures and SQ8 codes in 315.2 GiB local DRAM and stores raw vectors in CXL memory, whereas HNSW

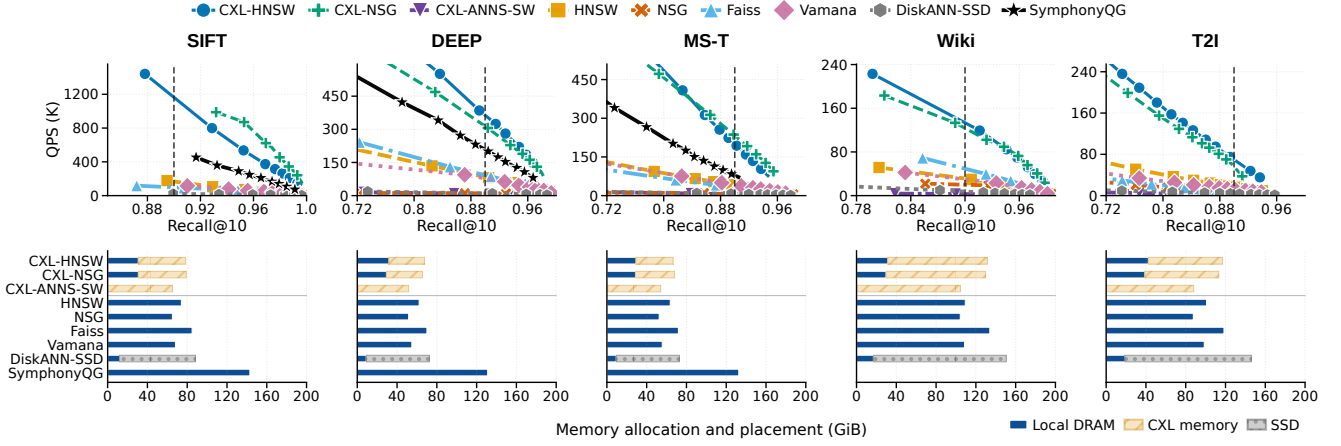


Figure 9: Comparison of the recall-QPS tradeoff and memory allocation and placement for five datasets.

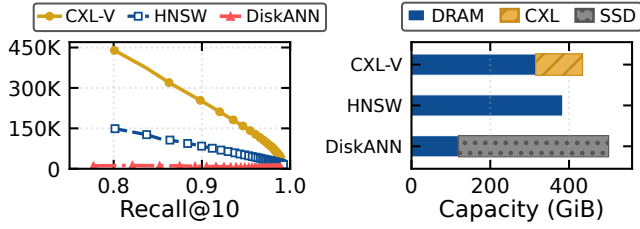


Figure 10: Billion-scale evaluation on BigANN.

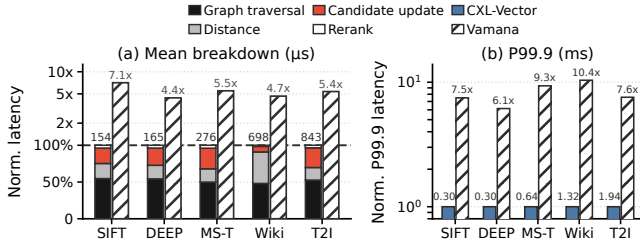


Figure 11: Mean and P99.9 latency.

keeps both graph and vectors in DRAM and DiskANN-SSD offloads the index to SSD. At $Recall@10$ (≥ 0.90), CXL-Vector outperforms HNSW by (3.0 $\times$) and DiskANN-SSD by (29.1 $\times$). Its local-DRAM footprint is 18% lower than HNSW’s because it stores only the actual neighbor entries for each node, whereas HNSW reserves a fixed-capacity neighbor list. Since SQ8 codes and the native uint8 vectors have the same byte width, and both systems use uint8 storage and distance computation, the speedup over HNSW comes from the execution design rather than quantization. DiskANN-SSD, by contrast, performs 86 4 KiB page reads per query (≈ 0.34 MiB). Although both systems move capacity beyond local DRAM, CXL’s byte-addressable interface avoids the block-I/O bottleneck.

Latency Analysis and Breakdown. Figure 11 evaluates average and tail latency under high-recall search. At 56 threads, CXL-Vector reduces mean latency by 4.4 $\times$ –7.1 $\times$ and P99.9 latency by 6.1 $\times$ –10.4 $\times$ across the evaluated datasets, demonstrating both higher efficiency and more stable concurrent execution. The latency breakdown further explains the improvement. After loop-split batching and the vectorized kernel, distance computation is no longer

the dominant cost, contributing only 17%–21% of latency on 96–200-dimensional datasets and 43% on Wikipedia with larger compact codes, compared with 50%–55% in the unoptimized tier-aware search. The latency is dominated by graph traversal and candidate management, while CXL accesses remain bounded through reranking.

6.3 Ablation Study

6.3.1 Isolating the Raw-Vector Capacity Tier. Figure 12 isolates the role of CXL as a capacity tier by comparing naive memory placement with CXL-Vector’s tier-aware organization. All configurations are evaluated at matched Recall@10 with 56 threads. We first consider naive HNSW placement: *All DRAM* keeps the entire index in local DRAM, *Interleave* evenly distributes the index across DRAM and CXL, and *All CXL* places the entire index in CXL memory. We further compare three CXL-Vector variants that differ only in the location of raw vectors (DRAM, CXL, or SSD) while keeping the same graph layout, compact codes, search algorithm, and reranking budget. Naive HNSW placement across DRAM and CXL (*All CXL* and *Interleave*) reduces memory usage but fails to preserve performance: *All CXL* is 3.4 $\times$ –11.3 $\times$ slower due to CXL bandwidth saturation, while *Interleave* achieves only 0.72 $\times$ –1.12 $\times$ the throughput of *All DRAM*. In contrast, CXL-Vector remains insensitive to the location of raw vectors: moving them from CXL to DRAM changes throughput by only -2.7% to $+0.8\%$, showing that its traversal path avoids capacity-tier bottlenecks. Replacing CXL with SSD causes a 3.4 $\times$ –24.2 $\times$ slowdown because reranking incurs page-granular I/O. Overall, CXL-Vector uses CXL as a capacity tier, reducing local DRAM usage by 50%–72% while preserving DRAM-like access behavior through its memory-aware search and execution design.

6.3.2 CXL Bandwidth Saturation Analysis. Figure 13 applies the bandwidth model in Equation (2) to identify whether CXL read bandwidth limits query throughput. We derive controller-observed CXL traffic from hardware counters and compare the consumed bandwidth λb_q against the measured saturation bandwidth (13.7 GiB/s). *All-CXL reaches the bandwidth ceiling.* As search width increases, *All-CXL* incurs substantially more CXL traffic: the per-query traffic grows by 4.36 $\times$ –6.18 $\times$ from $ef=16$ to $ef=128$. At the largest

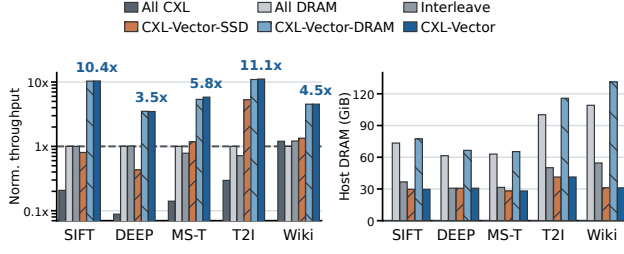


Figure 12: Isolating the raw-vector capacity tier.

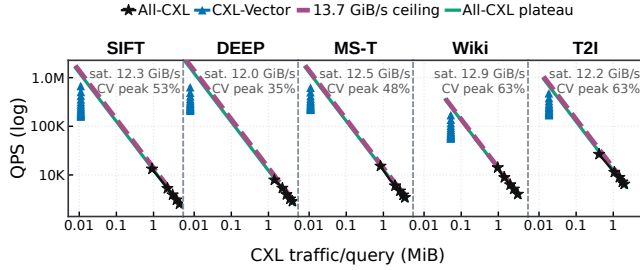


Figure 13: Bandwidth analysis.

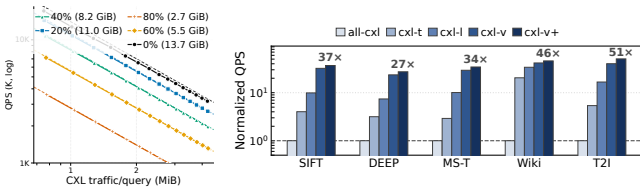


Figure 14: Bandwidth sensitivity.

Figure 15: Search pipeline optimization.

search width, all datasets consume 12.39–13.09 GiB/s, reaching 90.4%–95.5% of the measured CXL ceiling. This confirms that full-index CXL placement is ultimately bandwidth-bound: additional graph exploration increases bytes transferred rather than useful throughput. *CXL-Vector avoids CXL bandwidth saturation.* With raw-vector access confined to final reranking, controller counters report $b_q = 0.0078\text{--}0.0527$ MiB of physical CXL read traffic per query for CXL-Vector, which is $14.6\times\text{--}563.3\times$ lower than All-CXL under the same search width. Its peak CXL utilization remains at only $\rho_{\text{CXL}} = 0.354\text{--}0.634$, leaving sufficient bandwidth headroom for concurrent queries. These results verify that CXL-Vector converts CXL from an execution bottleneck into a capacity tier by controlling when and how raw vectors are accessed.

Bandwidth sensitivity. Figure 14 studies the impact of CXL bandwidth availability on All-CXL performance by applying different bandwidth caps (20%, 40%, 60%, and 80% of the measured peak bandwidth) on SIFT100M. As the available bandwidth decreases, All-CXL throughput degrades consistently because graph traversal and raw-vector accesses compete for the same shared CXL link. Under tighter bandwidth caps, the achieved CXL utilization approaches the corresponding bandwidth ceiling, indicating that the system becomes increasingly bandwidth-bound.

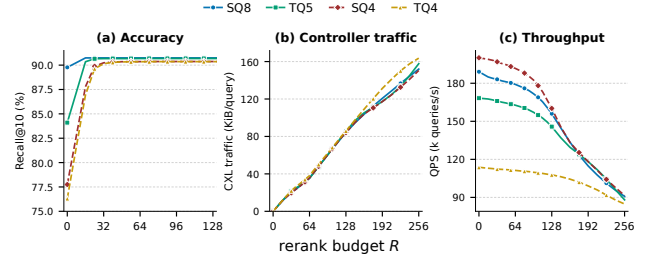


Figure 16: Impact of reranking budget.

6.3.3 Progressive Search Pipeline Optimization. Figure 15 presents the progressive optimization of the CXL-Vector search pipeline by cumulatively enabling individual mechanisms over the all-CXL baseline. Each stage targets a different bottleneck in graph ANNS execution, from memory placement to execution organization and kernel efficiency. Tier-aware search (cxi-t) contributes the largest single factor: $2.9\times\text{--}5.4\times$ on the four lower-dimensional datasets and $20.4\times$ on Wikipedia. The gain is largest on Wikipedia because its vectors generate the heaviest per-query CXL traffic under all-CXL placement; moving traversal off the CXL read path frees the most headroom there. Loop-split (cxi-l) adds $1.7\times\text{--}3.5\times$ by restructuring the traversal loop to expose batches of independent distance evaluations. SIMD vectorization (cxi-v) adds $1.2\times\text{--}3.2\times$ by processing those batches across vector lanes. VNNI (cxi-v+) contributes a further $1.10\times\text{--}1.26\times$ as an instruction-level optimization (§4.2.2). No stage regresses on any dataset, and the four mechanisms compound to a $27.2\times\text{--}50.5\times$ cumulative gain. Loop-split uses a fixed batch size of $B=16$, which saturates throughput because each expansion’s pending list is typically consumed in a single batch (Appendix F).

6.3.4 Effect of Bounded Reranking. The reranking budget R controls how many retained raw-vector records receive exact evaluation, trading additional verification for recall. Figure 16 measures the physical CXL traffic and throughput on MSTuring with $R = 0\text{--}256$ across four quantization settings. *Selected records and measured CXL traffic.* Since the retained pool is full at the evaluated operating points, $m_q = \min(R, |S|) = R$. For each quantizer, a linear fit of the controller-observed physical traffic b_q against R yields $R^2 = 0.993\text{--}0.997$; b_q increases from negligible traffic at $R = 0$ to $150\text{--}164$ KiB per query at $R = 256$. This near-linear empirical relationship shows that physical traffic closely tracks the selected-record count on our platform. *Recall-cost trade-off.* Increasing R improves recall by correcting approximate-distance misranking. High-fidelity SQ8 and TQ5 reach their recall ceiling at $R \approx 16\text{--}24$, while lower-bit SQ4 and TQ4 require $R \approx 48\text{--}64$. Beyond this point, additional exact evaluations provide diminishing recall improvement but continue to increase CXL traffic and computation cost. Therefore, we select SQ8 with a small reranking budget as the default configuration, achieving near-maximum recall with a small selected-record budget. A full comparison appears in Appendix C.

6.3.5 Scalable Query Execution. Concurrent Scalability. Figure 17 evaluates throughput scaling from 1 to 56 workers. Each configuration uses a fixed, dataset-specific search width achieving Recall@10 ≥ 0.90 . All-CXL fails to scale with concurrency because placing the entire index on CXL exposes fine-grained, data-dependent graph

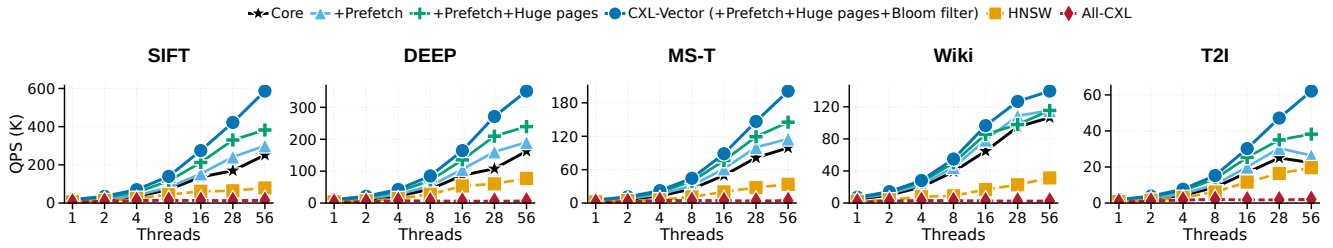


Figure 17: Thread scaling with cumulatively added prefetching, huge pages, and Bloom filtering.

traversal to the shared CXL bandwidth constraint. As concurrent queries increase, aggregate random reads quickly approach the device bandwidth ceiling (Figure 13), limiting additional parallelism. All-CXL saturates at 4–8 threads and achieves only $1.78\times$ – $3.51\times$ the single-thread throughput at 56 threads. In contrast, **CXL-Vector** confines traversal to local DRAM and bounds CXL accesses to the reranking stage, avoiding bandwidth saturation during concurrent search. This enables throughput scaling as concurrency increases and provides the foundation for further host-side optimizations evaluated below.

Optimization Impact. Figure 17 quantifies the cumulative impact of three optimizations. Starting from Core, which already combines tier-aware search and batched execution, **Chunk-ahead prefetching** improves throughput by $1.08\times$ – $1.19\times$ by hiding compact-code access latency. **Huge-page preparation** provides an additional $1.00\times$ – $1.45\times$ gain by reducing address-translation overhead, while the **Cache-resident Bloom filter** further improves throughput by $1.21\times$ – $1.63\times$ by alleviating visited-set contention. Together, these optimizations improve Core by $1.31\times$ – $2.78\times$ at 56 threads, enabling the complete CXL-Vector system to outperform HNSW by $3.17\times$ – $7.40\times$. These results demonstrate that bounding CXL traffic enables scalable execution, while the host-side optimizations effectively convert the available memory-system headroom into additional throughput. The cache-level, translation, and visited-set diagnostics appear in Appendix G.

7 Related Work

Graph-Based ANNS. Graph-based ANNS indexes provide one of the strongest recall–throughput tradeoffs among practical ANNS methods [4, 38, 54]. Representative designs include HNSW [36], NSG [13], and Vamana [48], along with earlier small-world graph indexes such as NSW [35], GNNS [24], and FANNG [25].

SSD-Based ANNS. SSD-based systems extend ANNS capacity by placing large portions of the index on storage. DiskANN [48] stores graph structures and compressed vector representations on SSD while using in-memory representations for traversal. SPANN [6] adopts a cluster-based organization with lightweight routing structures in memory. Follow-up studies improve locality [53], overlap I/O with computation [20], support dynamic updates [21, 55], and explore computational SSDs [50].

Memory-Tiered and Hardware-Accelerated ANNS. GPU-based ANNS systems exploit massive parallelism for distance computation [16, 30, 31, 41, 42, 51, 59], while near-memory approaches such as ANSMET [34] move computation closer to data. HM-ANN [47]

explores capacity scaling across memory tiers. Closest to our setting, CXL-ANNS [27] leverages CXL memory pools with endpoint-side acceleration for distance computation. However, it relies on specialized CXL hardware support, whereas CXL-Vector targets commodity memory-only CXL systems through a software-only execution redesign.

Quantization-Based ANNS. Vector quantization reduces vector storage cost while enabling efficient approximate distance evaluation, including product quantization [17, 29], anisotropic quantization [22], RaBitQ [14], and TurboQuant [57]. Practical systems such as Faiss IVF [11] and Milvus [52] use quantized codes for coarse filtering followed by refinement. For graph-based search, NGT-QG [28] and SymphonyQG [19] replicate quantized codes along graph edges to enable FastScan-style execution [2], trading memory footprint for SIMD efficiency. OG-LVQ [1] and VSAG [61] reduce this overhead through compact traversal representations and batched execution, but remain within a single memory tier. In contrast, CXL-Vector exposes batch parallelism through execution restructuring on unmodified graph topology and separates capacity scaling from traversal execution using DRAM–CXL tiering.

8 Conclusion

This work demonstrates that the key obstacle to exploiting CXL memory for graph-based ANNS is not merely its higher access latency, but the shared read bandwidth pressure caused by concurrent irregular accesses. Naive full-index CXL placement saturates this bandwidth, making throughput degrade with increasing bytes read per query. CXL-Vector addresses this challenge through a tier-aware execution model that separates traversal execution from capacity-tier accesses: graph search operates on local graph structures and compact representations, while CXL-resident vectors are accessed only for bounded candidate reranking. Beyond memory placement, CXL-Vector restructures graph execution through batched expansion and a vectorized distance kernel, exposing parallelism without increasing index footprint. Its scalable query execution design further mitigates host-side bottlenecks under high concurrency. We instantiate CXL-Vector on both HNSW and NSG without modifying their graph structures. At Recall@10 ≥ 0.90 , CXL-Vector achieves $3.9\times$ – $7.2\times$ the throughput of HNSW with 58% less local DRAM, outperforms SymphonyQG by $1.8\times$ – $2.9\times$ with 76%–78% less local DRAM, and sustains a $29.1\times$ throughput advantage over DiskANN-SSD at billion scale. Compared with full-index CXL placement, CXL-Vector reduces CXL read traffic by $14.6\times$ – $563.3\times$.

References

- [1] Cecilia Aguerrebere, Ishwar Singh Bhati, Mark Hildebrand, Mariano Tepper, and Theodore Willke. 2023. Similarity Search in the Blink of an Eye with Compressed Indices. *Proc. VLDB Endow.* 16, 11 (July 2023), 3433–3446. doi:10.14778/3611479.3611537
- [2] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2015. Cache locality is not enough: high-performance nearest neighbor search with product quantization fast scan. *Proc. VLDB Endow.* 9, 4 (Dec. 2015), 288–299. doi:10.14778/2856318.2856324
- [3] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2021. Quicker ADC : Unlocking the Hidden Potential of Product Quantization With SIMD. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 43, 5 (2021), 1666–1677. doi:10.1109/TPAMI.2019.2952606
- [4] Martin Aumüller, Erik Bernhardsson, and Alexander John Faithfull. 2020. ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms. *Information Systems* 87 (2020), 101374. doi:10.1016/j.is.2019.02.006
- [5] Jon Louis Bentley. 1975. Multidimensional binary search trees used for associative searching. *Commun. ACM* 18, 9 (Sept. 1975), 509–517. doi:10.1145/361002.361007
- [6] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: highly-efficient billion-scale approximate nearest neighbor search. In *Proceedings of the 35th International Conference on Neural Information Processing Systems (NIPS '21)*. Curran Associates Inc., Red Hook, NY, USA, Article 398, 14 pages.
- [7] Yue Chen, Kai Zhang, Sipeng Chen, Shihai Xiao, Xiaomin Zou, Ren Ren, Yinan Jing, X. Sean Wang, Li Cao, and Mingxiang Wan. 2025. RED-ANNS: An RDMA-Enabled Distributed Framework for Graph-Based Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 19, 3 (Nov. 2025), 399–412. doi:10.14778/3778092.3778101
- [8] Rongxin Cheng, Yifan Peng, Xingda Wei, Hongrui Xie, Rong Chen, Sijie Shen, and Haibo Chen. 2024. Characterizing the Dilemma of Performance and Index Size in Billion-Scale Vector Search and Breaking It with Second-Tier Memory. arXiv:2405.03267 [cs.DC] doi:10.48550/arXiv.2405.03267
- [9] Debendra Das Sharma, Robert Blankenship, and Daniel Berger. 2024. An Introduction to the Compute Express Link (CXL) Interconnect. *ACM Comput. Surv.* 56, 11, Article 290 (July 2024), 37 pages. doi:10.1145/3669900
- [10] Sanjoy Dasgupta and Yoav Freund. 2008. Random projection trees and low dimensional manifolds. In *Proceedings of the Fortieth Annual ACM Symposium on Theory of Computing* (Victoria, British Columbia, Canada) (STOC '08). Association for Computing Machinery, New York, NY, USA, 537–546. doi:10.1145/1374376.1374452
- [11] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2026. The Faiss Library. *IEEE Transactions on Big Data* 12, 2 (2026), 346–361. doi:10.1109/TBDATA.2025.3618474
- [12] Fei Fang, Yi Liu, and Chen Qian. 2026. d-HNSW: A High-performance Vector Search Engine on Disaggregated Memory. In *Abstracts of the 2026 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems (USA) (SIGMETRICS Abstracts '26)*. Association for Computing Machinery, New York, NY, USA, 229–231. doi:10.1145/3801489.3806852
- [13] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast approximate nearest neighbor search with the navigating spreading-out graph. *Proc. VLDB Endow.* 12, 5 (Jan. 2019), 461–474. doi:10.14778/3303753.3303754
- [14] Jianyang Gao and Cheng Long. 2024. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 3, Article 167 (May 2024), 27 pages. doi:10.1145/3654970
- [15] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. 2023. Precise Zero-Shot Dense Retrieval without Relevance Labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 1762–1777. doi:10.18653/v1/2023.acl-long.99
- [16] Vincent Garcia, Eric Debreuve, and Michel Barlaud. 2008. Fast k Nearest Neighbor Search using GPU. arXiv:0804.1448 [cs.CV] https://arxiv.org/abs/0804.1448
- [17] Tiezheng Ge, Kaiping He, Qifa Ke, and Jian Sun. 2013. Optimized Product Quantization for Approximate Nearest Neighbor Search. In *Proceedings of the 2013 IEEE Conference on Computer Vision and Pattern Recognition (CVPR '13)*. IEEE Computer Society, USA, 2946–2953. doi:10.1109/CVPR.2013.379
- [18] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 1999. Similarity Search in High Dimensions via Hashing. In *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB '99)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 518–529.
- [19] Yutong Gou, Jianyang Gao, Yuxuan Xu, and Cheng Long. 2025. SymphonyQG: Towards Symphonious Integration of Quantization and Graph for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 1, Article 80 (Feb. 2025), 26 pages. doi:10.1145/3709730
- [20] Hao Guo and Youyou Lu. 2026. Achieving Low-Latency Graph-Based Vector Search via Aligning Best-First Search Algorithm with SSD. *ACM Trans. Storage* 22, 2, Article 12 (March 2026), 30 pages. doi:10.1145/3793926
- [21] Hao Guo and Youyou Lu. 2026. OdinANN: Direct Insert for Consistently Stable Performance in Billion-Scale Graph-Based Vector Search. In *24th USENIX Conference on File and Storage Technologies (FAST 26)*. USENIX Association, Santa Clara, CA, 133–147. https://www.usenix.org/conference/fast26/presentation/guo
- [22] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, Virtual Event, 3887–3896. https://proceedings.mlr.press/v119/guo20h.html
- [23] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. REALM: retrieval-augmented language model pre-training. In *Proceedings of the 37th International Conference on Machine Learning (ICML '20)*. JMLR.org, Article 368, 10 pages.
- [24] Kiana Hajebi, Yasin Abbasi-Yadkori, Hossein Shahbazi, and Hong Zhang. 2011. Fast approximate nearest-neighbor search with k-nearest neighbor graph. In *Proceedings of the Twenty-Second International Joint Conference on Artificial Intelligence - Volume Volume Two* (Barcelona, Catalonia, Spain) (IJCAI '11). AAAI Press, 1312–1317.
- [25] Ben Harwood and Tom Drummond. 2016. FANNG: Fast Approximate Nearest Neighbour Graphs. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 5713–5722. doi:10.1109/CVPR.2016.616
- [26] Piotr Indyk and Rajeev Motwani. 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing* (Dallas, Texas, USA) (STOC '98). Association for Computing Machinery, New York, NY, USA, 604–613. doi:10.1145/276698.276876
- [27] Junhyeok Jeong, Hanjin Choi, Hanyeoreum Bae, Seungjun Lee, Miryeong Kwon, and Myoungsoo Jung. 2023. CXL-ANNS: Software-Hardware Collaborative Memory Disaggregation and Computation for Billion-Scale Approximate Nearest Neighbor Search. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, Boston, MA, 585–600. https://www.usenix.org/conference/atc23/presentation/jang
- [28] Yahoo Japan. 2018. Neighborhood Graph and Tree for Indexing High-dimensional Data. https://github.com/yahoojapan/NGT. Accessed: 17 Apr. 2024.
- [29] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Trans. Pattern Anal. Mach. Intell.* 33, 1 (Jan. 2011), 117–128. doi:10.1109/TPAMI.2010.57
- [30] Haodi Jiang, Hao Guo, Minhui Xie, Jiwei Shu, and Youyou Lu. 2025. High-Throughput, Cost-Effective Billion-Scale Vector Search with a Single GPU. *Proc. ACM Manag. Data* 3, 6, Article 334 (Dec. 2025), 27 pages. doi:10.1145/3769799
- [31] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547. doi:10.1109/TBDATA.2019.2921572
- [32] Dingyi Kang, Juncheng Yang, and Bingzhe Li. 2026. LAANN: I/O-Aware Look-Ahead Search for Disk-Based Approximate Nearest Neighbor Search. arXiv:2606.02784 [cs.DB] https://arxiv.org/abs/2606.02784
- [33] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) (NIPS '20). Curran Associates Inc., Red Hook, NY, USA, Article 793, 16 pages.
- [34] Yiwei Li, Yuxin Jin, Boyu Tian, Huanchen Zhang, and Mingyu Gao. 2025. ANS-MET: Approximate Nearest Neighbor Search with Near-Memory Processing and Hybrid Early Termination. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery, New York, NY, USA, 1093–1107. doi:10.1145/3695053.3731013
- [35] Yuri Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems* 45 (2014), 61–68. doi:10.1016/j.is.2013.10.006
- [36] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (April 2020), 824–836. doi:10.1109/TPAMI.2018.2889473
- [37] Aninda Manocha, Zi Yan, Esin Tureci, Juan Luis Aragón, David Nellans, and Margaret Martonosi. 2022. The Implications of Page Size Management on Graph Analytics. In *2022 IEEE International Symposium on Workload Characterization (IISWC)*. 199–214. doi:10.1109/IISWC55918.2022.00026
- [38] Magdalen Dobson Manohar, Zheqi Shen, Guy Blelloch, Laxman Dhulipala, Yan Gu, Harsha Vardhan Simhadri, and Yihan Sun. 2024. ParlayANN: Scalable and Deterministic Parallel Graph-Based Approximate Nearest Neighbor Search Algorithms. In *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming* (Edinburgh, United Kingdom) (PPoPP '24). Association for Computing Machinery, New York, NY, USA, 270–285.

- doi:10.1145/3627535.3638475
- [39] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient Estimation of Word Representations in Vector Space. arXiv:1301.3781 [cs.CL] <https://arxiv.org/abs/1301.3781>
- [40] Jason Mohoney, Devesh Sarda, Mengze Tang, Shihabur Rahman Chowdhury, Anil Pacaci, Ihab F. Ilyas, Theodoros Rekatsinas, and Shivaram Venkataraman. 2025. Quake: adaptive indexing for vector search. In *Proceedings of the 19th USENIX Conference on Operating Systems Design and Implementation* (Boston, MA, USA) (OSDI '25). USENIX Association, USA, Article 9, 17 pages.
- [41] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. 2024. CAGRA: Highly Parallel Graph Construction and Approximate Nearest Neighbor Search for GPUs. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE Computer Society, Los Alamitos, CA, USA, 4236–4247. doi:10.1109/ICDE60146.2024.00323
- [42] Yuchen Peng, Dingyu Yang, Zhongle Xie, Ji Sun, Lidan Shou, Ke Chen, and Gang Chen. 2026. SVFusion: A CPU-GPU Co-Processing Architecture for Large-Scale Real-Time Vector Search. *Proc. VLDB Endow.* 19, 5 (May 2026), 1074–1087. doi:10.14778/3796195.3796216
- [43] Jeffrey Pennington, Richard Socher, and Christopher Manning. 2014. GloVe: Global Vectors for Word Representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Alessandro Moschitti, Bo Pang, and Walter Daelemans (Eds.). Association for Computational Linguistics, Doha, Qatar, 1532–1543. doi:10.3115/v1/D14-1162
- [44] Orestis Polychroniou, Arun Raghavan, and Kenneth A. Ross. 2015. Rethinking SIMD Vectorization for In-Memory Databases. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data* (Melbourne, Victoria, Australia) (SIGMOD '15). Association for Computing Machinery, New York, NY, USA, 1493–1508. doi:10.1145/2723372.2747645
- [45] Felix Putze, Peter Sanders, and Johannes Singler. 2010. Cache-, hash-, and space-efficient bloom filters. *ACM J. Exp. Algorithmics* 14, Article 4 (Jan. 2010), 18 pages. doi:10.1145/1498698.1594230
- [46] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. arXiv:2103.00020 [cs.CV] <https://arxiv.org/abs/2103.00020>
- [47] Jie Ren, Minjia Zhang, and Dong Li. 2020. HM-ANN: efficient billion-point nearest neighbor search on heterogeneous memory. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) (NIPS '20). Curran Associates Inc., Red Hook, NY, USA, Article 895, 13 pages.
- [48] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2019. *DiskANN: fast accurate billion-point nearest neighbor search on a single node*. Curran Associates Inc., Red Hook, NY, USA.
- [49] Yan Sun, Yifan Yuan, Zeduo Yu, Reese Kuper, Chihun Song, Jinghan Huang, Houxiang Ji, Siddharth Agarwal, Jiaqi Lou, Ipoom Jeong, Ren Wang, Jung Ho Ahn, Tianyin Xu, and Nam Sung Kim. 2023. Demystifying CXL Memory with Genuine CXL-Ready Systems and Devices. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2023*. ACM, Toronto, ON, Canada, 105–121. doi:10.1145/3613424.3614256
- [50] Bing Tian, Haikun Liu, Zhuohui Duan, Xiaofei Liao, Hai Jin, and Yu Zhang. 2024. Scalable Billion-point Approximate Nearest Neighbor Search Using SmartSSDs. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. USENIX Association, Santa Clara, CA, 1135–1150. <https://www.usenix.org/conference/atc24/presentation/tian>
- [51] Bing Tian, Haikun Liu, Yuhang Tang, Shihai Xiao, Zhuohui Duan, Xiaofei Liao, Hai Jin, Xuechang Zhang, Junhua Zhu, and Yu Zhang. 2025. Towards High-throughput and Low-latency Billion-scale Vector Search via CPU/GPU Collaborative Filtering and Re-ranking. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. USENIX Association, Santa Clara, CA, 171–185. <https://www.usenix.org/conference/fast25/presentation/tian-bing>
- [52] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data* (Virtual Event, China) (SIGMOD '21). Association for Computing Machinery, New York, NY, USA, 2614–2627. doi:10.1145/3448016.3457550
- [53] Mengzhao Wang, Weizhi Xu, Xiaomeng Yi, Songlin Wu, Zhangyang Peng, Xiangyu Ke, Yunjun Gao, Xiaoliang Xu, Rentong Guo, and Charles Xie. 2024. Starling: An I/O-Efficient Disk-Resident Graph Index Framework for High-Dimensional Vector Similarity Search on Data Segment. *Proceedings of the ACM on Management of Data* 2, 1, Article 14 (2024), 27 pages. doi:10.1145/3639269
- [54] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. *Proc. VLDB Endow.* 14, 11 (July 2021), 1964–1978. doi:10.14778/3476249.3476255
- [55] Yuming Xu, Hengyu Liang, Jin Li, Shuotao Xu, Qi Chen, Qianxi Zhang, Cheng Li, Ziyue Yang, Fan Yang, Yuqing Yang, Peng Cheng, and Mao Yang. 2023. SPFresh: Incremental In-Place Update for Billion-Scale Vector Search. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) (SOSP '23). Association for Computing Machinery, New York, NY, USA, 545–561. doi:10.1145/3600006.3613166
- [56] Xinjun Yang, Yingqiang Zhang, Hao Chen, Feifei Li, Gerry Fan, Yang Kong, Bo Wang, Jing Fang, Yuhui Wang, Tao Huang, Wenpu Hu, Jim Kao, and Jianping Jiang. 2025. Unlocking the Potential of CXL for Disaggregated Memory in Cloud-Native Databases. In *Companion of the 2025 International Conference on Management of Data* (Berlin, Germany) (SIGMOD/PODS '25). Association for Computing Machinery, New York, NY, USA, 689–702. doi:10.1145/3722212.3724460
- [57] Amir Zandieh, Majid Daliri, Majid Hadian, and Vahab Mirrokni. 2025. TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate. arXiv:2504.19874 [cs.LG] <https://arxiv.org/abs/2504.19874>
- [58] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, Mao Yang, and Lidong Zhou. 2023. VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. USENIX Association, Boston, MA, 377–395. <https://www.usenix.org/conference/osdi23/presentation/zhang-qianxi>
- [59] Zili Zhang, Fangyue Liu, Gang Huang, Xuanzhe Liu, and Xin Jin. 2024. Fast Vector Query Processing for Large Datasets Beyond GPU Memory with Re-ordered Pipelining. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. USENIX Association, Santa Clara, CA, 23–40. <https://www.usenix.org/conference/nsdi24/presentation/zhang-zili-pipelining>
- [60] Xiangyu Zhi, Meng Chen, Xiao Yan, Baotong Lu, Hui Li, Qianxi Zhang, Qi Chen, and James Cheng. 2026. CoTra: Towards Efficient and Scalable Distributed Vector Search with RDMA. *Proc. ACM Manag. Data* 4, 1, Article 20 (April 2026), 27 pages. doi:10.1145/3786634
- [61] Xiaoyao Zhong, Haotian Li, Jiabao Jin, Mingyu Yang, Deming Chu, Xiangyu Wang, Zhitao Shen, Wei Jia, George Gu, Yi Xie, Xuemin Lin, Heng Tao Shen, Jingkuan Song, and Peng Cheng. 2025. VSAG: An Optimized Search Framework for Graph-Based Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 18, 12 (Aug. 2025), 5017–5030. doi:10.14778/3750601.3750624